

Guía paso a paso: descargar el material del curso desde GitHub y trabajar en Visual Studio Code

Programación y Base de Datos · Carrera de Ingeniería Ambiental · Universidad Nacional de Loja ·
Septiembre 2026 – Febrero 2027

Docente: Carlos Guillermo Chunchu Morocho · Repositorio: https://github.com/cguillermo79/Informacion_progra

Índice

1. ¿Qué vamos a lograr?	1
2. Preparar el lugar de trabajo	2
3. Verificar y configurar las herramientas	4
4. Descargar el material del curso	5
5. Entorno virtual y bibliotecas	5
6. Comprobar que Python lee la base de su equipo	6
7. Cada vez que abra el proyecto	8
8. Guardar su trabajo con Git (commits)	8
9. Problemas frecuentes	9
10. Hoja de referencia rápida	10
11. Reglas importantes	10

1. ¿Qué vamos a lograr?

Al terminar esta guía usted habrá:

1. Creado una **carpeta de trabajo** en su computadora.
2. **Descargado** desde GitHub todo el material del curso (cuadernos, guías y bases de datos) con un solo comando.
3. Aprendido a **actualizar** ese material cada vez que el docente agregue contenido.
4. Creado un **entorno virtual** (.venv) e **instalado las bibliotecas** que usaremos.
5. Comprobado que Python **lee la base de datos de su equipo**.
6. Aprendido a guardar su propio trabajo con **Git** (commit).

No necesita saber programar ni tener cuenta de GitHub. Solo debe seguir los pasos **en orden**, sin saltarse ninguno. Si un paso falla, vaya a la sección «**Problemas frecuentes**» al final.

Requisito previo: haber instalado **Visual Studio Code**, **Python** y **Git** con la *Guía de instalación y configuración del entorno* (carpeta 00_guia_instalacion). En el Paso 5 comprobaremos que todo quedó bien instalado.

1.1 Cómo quedará organizada su computadora

Al final tendrá esta estructura (la iremos creando paso a paso):

```
C:\Programacion\          <- carpeta principal (la crea usted en el Paso 1)
|-- Informacion_programacion\  <- material del docente (se descarga con Git; NO se
    ↪ edita)
|-- mis_trabajos\           <- SU trabajo: aquí copia y resuelve los cuadernos
`-- .venv\                  <- entorno virtual de Python (se crea en el Paso 9)
```

Regla de oro: la carpeta `Informacion_programacion` es **solo para recibir material**. Si usted modifica archivos dentro de ella, después Git no podrá actualizarla. **Siempre copie** un cuaderno a `mis_trabajos` antes de resolverlo.

1.2 Palabras que va a escuchar

Palabra	Significado simple
Terminal	Una ventana donde usted escribe órdenes en lugar de hacer clic. Usaremos la que viene dentro de Visual Studio Code.
Comando	Una orden escrita en la terminal. Se ejecuta al presionar Enter .
Git	Programa que descarga y registra los cambios de los archivos.
GitHub	Página web donde el docente guarda el material del curso.
Repositorio	La carpeta del curso en GitHub (contiene todo el material).
Clonar	Descargar por primera vez el repositorio. Se hace una sola vez .
<code>git pull</code>	Traer lo nuevo que el docente subió. Se hace cada vez que abra el proyecto.
Commit	Una «foto» de su trabajo con un mensaje. Sirve para guardar versiones de su trabajo.
Entorno virtual (.venv)	Una carpeta con su propia copia de Python y de las bibliotecas del curso, para que nada se mezcle con otros programas.
Biblioteca	Código ya hecho que usamos en Python (por ejemplo pandas, para leer tablas).
<code>pip</code>	El instalador de bibliotecas de Python.

2. Preparar el lugar de trabajo

Paso 1. Crear la carpeta Programacion (sin escribir comandos)

1. Abra el **Explorador de archivos** de Windows (el icono de la carpeta amarilla en la barra de tareas, o presione las teclas **Windows + E**).
2. En el panel izquierdo haga clic en **Este equipo** y luego doble clic en **Disco local (C:)**.
3. Haga **clic derecho** en un espacio vacío → **Nuevo** → **Carpeta**.
4. Escriba el nombre **Programacion** (sin tilde y sin espacios) y presione **Enter**.

Ahora existe la carpeta `C:\Programacion`.

¿Por qué en `C:\` y sin tildes ni espacios? Las rutas cortas y sin caracteres especiales evitan muchos errores de Git y Python. Evite también carpetas sincronizadas con OneDrive.

Paso 2. Abrir esa carpeta en Visual Studio Code

1. Abra **Visual Studio Code**.
2. En el menú superior haga clic en **Archivo** (en inglés: *File*) → **Abrir carpeta...** (*Open Folder...*).
3. Busque y seleccione **C:\Programacion** y pulse el botón **Seleccionar carpeta**.
4. Si aparece la pregunta «¿Confía en los autores de los archivos de esta carpeta?», pulse **Sí, confío en los autores**.

Verá en la parte superior izquierda, en el panel **Explorador**, el nombre **PROGRAMACION** (vacío por ahora).

Siempre que trabaje en el curso, abra **esta misma carpeta** (C:\Programacion) en Visual Studio Code.

Paso 3. Abrir la terminal dentro de Visual Studio Code

1. En el menú superior haga clic en **Terminal** → **Nueva terminal** (en inglés: *Terminal* → *New Terminal*).
2. Se abre un panel en la **parte inferior** de la ventana. Esa es la terminal.
3. Verá algo parecido a esto:

```
PS C:\Programacion>
```

Lo que significa:

- **PS** indica que es **PowerShell**, el tipo de terminal que usaremos.
- **C:\Programacion** es la **carpeta actual**: los comandos se ejecutan «en esa carpeta». Como abrimos C:\Programacion en el Paso 2, la terminal ya está ahí.
- **>** y el cursor que parpadea indican **dónde debe escribir**.

Si en la esquina superior derecha del panel no dice **powershell**, pulse la pequeña flecha junto al signo **+** y elija **PowerShell**.

Paso 4. Aprender a escribir en la terminal

Cómo se escribe un comando:

1. Haga **clic dentro** del panel de la terminal (así aparece el cursor).
2. Escriba (o pegue con **clic derecho** o **Ctrl + V**) el comando.
3. Presione **Enter** para ejecutarlo.
4. Espere a que vuelva a aparecer una línea que termina en **>**; entonces el comando terminó.

Practique ahora escribiendo estos comandos, uno por vez:

```
pwd
```

Muestra la carpeta actual. Debe responder C:\Programacion.

```
dir
```

Muestra los archivos de la carpeta (por ahora está vacía).

Otros comandos que le serán útiles:

Comando	Qué hace
cd nombre	Entra a la carpeta <i>nombre</i> .
cd ..	Sube a la carpeta anterior.
cls	Limpia la pantalla de la terminal.
Flecha ↑ del teclado	Escribe de nuevo el comando anterior.

Comando	Qué hace
---------	----------

Tecla Tab	Completa automáticamente el nombre de una carpeta o archivo.
------------------	--

Si algo sale en letras rojas: no se asuste. Lea el mensaje, revise que escribió el comando exactamente igual y consulte la sección «Problemas frecuentes». **Un error no daña su computadora.**

3. Verificar y configurar las herramientas

Paso 5. Comprobar que Git y Python están instalados

En la terminal de Visual Studio Code escriba **cada línea** y presione **Enter** después de cada una:

```
git --version
```

Debe responder algo como `git version 2.45.0.windows.1`.

```
python --version
```

Debe responder algo como `Python 3.12.4` (use **Python 3.10 o superior**).

```
pip --version
```

Debe responder algo como `pip 24.0 from C:\...\site-packages\pip (python 3.12)`.

Si en alguno aparece un error:

Mensaje	Qué significa y qué hacer
git : El término 'git' no se reconoce...	Git no está instalado. Descárguelo de https://git-scm.com/download/win , instálelo aceptando las opciones por defecto, cierre Visual Studio Code completamente , ábralo otra vez y repita.
Python was not found; run without arguments to install from the Microsoft Store	Python no está instalado. Instálelo de https://www.python.org/downloads/ y marque la casilla «Add python.exe to PATH» antes de pulsar <i>Install Now</i> . Cierre y abra Visual Studio Code.
python : El término 'python' no se reconoce...	Python se instaló sin «Add to PATH». Pruebe con <code>py --version</code> ; si funciona, escriba <code>py</code> donde esta guía diga <code>python</code> . Si no, reinstale marcando la casilla.

Paso 6. Decirle a Git quién es usted (solo la primera vez)

Escriba estas dos líneas **cambiando lo que está entre comillas por su nombre y su correo**:

```
git config --global user.name "Nombre Apellido"
git config --global user.email "su_correo@gmail.com"
```

Compruebe que se guardó:

```
git config --global --list
```

Debe ver en la lista `user.name=` y `user.email=` con sus datos.

4. Descargar el material del curso

Paso 7. Clonar (descargar) el repositorio

Compruebe primero que la terminal está en C:\Programacion (escriba pwd). Luego escriba **esta línea completa**:

```
git clone https://github.com/cguillermo79/Informacion_programacion.git
```

Verá mensajes como Cloning into 'Informacion_programacion'... y porcentajes de descarga. **Espere** hasta que vuelva a aparecer PS C:\Programacion>.

Compruebe que se descargó:

```
dir
```

Debe aparecer la carpeta **Informacion_programacion**. También la verá en el panel **Explorador** de la izquierda (si no aparece, pulse el icono de *actualizar* del Explorador).

Dentro encontrará:

- 01_unidad_1, 02_unidad_2, 03_unidad_3: material de cada unidad.
- bases_datos_proyecto_integrador: las bases de datos de los equipos.
- README.md y requirements.txt.

Las bases de datos pesan **pocos megabytes** (unos 10 a 15 MB cada una). La descarga completa tarda solo unos instantes.

Esto se hace una sola vez. Si lo repite le dirá destination path already exists: es normal, significa que ya lo descargó. Para tener lo nuevo use git pull (Paso 15).

5. Entorno virtual y bibliotecas

Paso 8. Instalar las extensiones de Visual Studio Code

1. En la barra izquierda de Visual Studio Code haga clic en el icono de **Extensiones** (cuatro cuadritos) o presione **Ctrl + Shift + X**.
2. En el buscador escriba **Python**, elija la de **Microsoft** y pulse **Instalar**.
3. Repita con **Jupyter** (también de Microsoft).

Paso 9. Crear el entorno virtual .venv (solo la primera vez)

Compruebe que la terminal está en C:\Programacion (pwd) y escriba:

```
python -m venv .venv
```

Tarda unos segundos y **no muestra ningún mensaje**. Compruebe:

```
dir -Force
```

Debe aparecer una carpeta llamada **.venv** (el -Force sirve para ver carpetas ocultas).

Paso 10. Activar el entorno virtual

Escriba:

```
.\.venv\Scripts\Activate.ps1
```

Si funcionó, la línea de la terminal ahora empieza con (**.venv**):

(.venv) PS C:\Programacion>

Eso significa que el entorno está **activo**. Debe verse (**.venv**) siempre que instale algo o ejecute programas del curso.

Si aparece un error en rojo que dice «la ejecución de scripts está deshabilitada en este sistema», escriba **una sola vez** este comando:

`Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned`

Cuando pregunte, escriba **S** y presione **Enter**. Luego repita el comando de activación.

Para **desactivar** el entorno (cuando termine de trabajar) escriba `deactivate`.

Paso 11. Decirle a Visual Studio Code que use ese entorno

1. Presione **Ctrl + Shift + P** (se abre una barra de búsqueda en la parte superior).
2. Escriba **Python: Select Interpreter** y presione **Enter**.
3. En la lista elija la opción que dice **.venv** (termina en `.venv\Scripts\python.exe`).

En la esquina inferior derecha de la ventana debe aparecer el número de versión de Python seguido de (`' .venv' : venv`).

Paso 12. Instalar las bibliotecas del curso

Con (**.venv**) visible en la terminal, escriba estas dos líneas, una por vez:

```
python -m pip install --upgrade pip
```

```
pip install -r Informacion_programacion\requirements.txt
```

La segunda puede tardar **varios minutos** y mostrar muchas líneas: es normal. Termina cuando vuelve a aparecer (**.venv**) PS C:\Programacion>.

Compruebe que quedó bien:

```
python -c "import pandas, matplotlib; print('Todo listo')"
```

Debe responder **Todo listo**. (sqlite3, que usaremos en SQL, ya viene con Python.)

6. Comprobar que Python lee la base de su equipo

Paso 13. Su carpeta de trabajo y el archivo de prueba

Crear la carpeta mis_trabajos dentro de Visual Studio Code (sin comandos):

1. En el panel **Explorador** (izquierda), pase el mouse sobre el nombre **PROGRAMACION**: aparecen unos iconos pequeños.
2. Haga clic en el icono de **Nueva carpeta** y escriba **mis_trabajos**, luego **Enter**.

Crear el archivo de prueba:

1. Haga clic derecho sobre la carpeta **mis_trabajos** → **Nuevo archivo...**
2. Escriba el nombre **test_datos.py** y presione **Enter**. Se abre un archivo en blanco en el centro de la pantalla.
3. **Copie y pegue** este código dentro del archivo:

```
import pandas as pd

# CAMBIE SOLO ESTA LINEA por la carpeta de su equipo (ver lista abajo)
EQUIPO = "equipo_01_relieve_zona_norte"

ruta = (
    "Informacion_programacion/bases_datos_proyecto_integrador/"
    f"{EQUIPO}/base_programacion_{EQUIPO}_2019_2024.csv"
)

df = pd.read_csv(ruta)
print("Lectura exitosa")
print("Filas:", df.shape[0], "| Columnas:", df.shape[1])
print("Celdas unicas:", df["cell_id"].nunique())
print(df.head(3))
```

4. Guarde con **Ctrl + S**.

La línea **EQUIPO = ...** debe tener el nombre de la carpeta de su equipo:

- Equipo 1: equipo_01_relieve_zona_norte
- Equipo 2: equipo_02_relieve_zona_sur
- Equipo 3: equipo_03_cobertura_loja_completo
- Equipo 4: equipo_04_clima_zona_norte
- Equipo 5: equipo_05_clima_zona_sur
- Equipo 6: equipo_06_incendios_loja_completo

Ejecutar la prueba. En la terminal (con (.venv) visible y en C:\Programacion) escriba:

```
python mis_trabajos\test_datos.py
```

Resultado esperado según su equipo:

- Equipos **1, 2, 4 y 5**: Filas: 18000, Celdas unicas: 250.
- Equipos **3 y 6**: Filas: 36000, Celdas unicas: 500.
- Columnas: equipos 1 y 2 → **66**; equipo 3 → **53**; equipos 4 y 5 → **57**; equipo 6 → **41**.

Si coincide, **su entorno está listo**. Si aparece `FileNotFoundError`, el nombre de la carpeta en **EQUIPO** está mal escrito o la terminal no está en `C:\Programacion` (use `pwd`).

Use **únicamente** la base de su equipo. Las bases son de **solo lectura**: nunca las edite ni las guarde de nuevo.

Paso 14. Copiar un cuaderno y trabajar en él

1. En el Explorador de Visual Studio Code abra: `Informacion_programacion` → `01_unidad_1` → `02_micro_retos`.
2. Haga clic derecho sobre el cuaderno (por ejemplo `MR1_errores_de_programacion.ipynb`) → **Copiar**.
3. Haga clic derecho sobre la carpeta `mis_trabajos` → **Pegar**.
4. Abra la copia que está en `mis_trabajos` (doble clic).
5. Arriba a la derecha pulse **Seleccionar kernel** (*Select Kernel*) → **Python Environments...** → elija `.venv`.
6. Ejecute una celda con **Shift + Enter** y guarde con **Ctrl + S**.

Trabaje siempre en la copia de `mis_trabajos`, nunca en el original.

7. Cada vez que abra el proyecto

Haga estos pasos **siempre, en este orden**:

1. Abra **Visual Studio Code** → **Archivo** → **Abrir carpeta** → **C:\Programacion** (suele aparecer en *Abrir reciente*).
2. Abra la terminal: **Terminal** → **Nueva terminal**.
3. **Active el entorno virtual** (debe verse (.venv)):

```
.\.venv\Scripts\Activate.ps1
```

Paso 15. Traer lo nuevo del docente

4. Escriba:

```
git -C Informacion_programacion pull
```

Puede responder:

- Already up to date. → no hay novedades.
- Una lista de archivos con signos + y - → se descargó material nuevo; búsquelo en la carpeta de la unidad.

5. Si el docente avisa que cambió requirements.txt, vuelva a ejecutar:

```
pip install -r Informacion_programacion\requirements.txt
```

6. Abra su cuaderno de mis_trabajos, elija el kernel .venv y trabaje.

Si **git pull** dice «**Your local changes ... would be overwritten**»: modificó un archivo dentro de Informacion_programacion. Escriba `git -C Informacion_programacion status`: verá los archivos modificados. **Copie a mis_trabajos los que tengan trabajo suyo** y luego escriba (cambiando la ruta por la del archivo que aparece):

```
git -C Informacion_programacion restore ruta\del\archivo
```

```
git -C Informacion_programacion pull
```

restore descarta sus cambios en ese archivo: **primero guarde una copia.**

8. Guardar su trabajo con Git (commits)

El repositorio del docente es de **solo descarga**: usted no sube nada allí. Pero puede llevar el control de versiones de **su propio trabajo** en mis_trabajos.

Paso 16. Crear su repositorio personal (solo una vez)

En la terminal (con (.venv) y en C:\Programacion):

```
cd mis_trabajos
git init
```

Verá Initialized empty Git repository. La terminal ahora dice ... \mis_trabajos>.

Cree el archivo .gitignore para que Git **no guarde** datos pesados: en el Explorador, clic derecho sobre mis_trabajos → **Nuevo archivo...** → nombre **.gitignore** y pegue:

```
__pycache__/  
.ipynb_checkpoints/
```

```
*.csv  
resultados/
```

Guarde con **Ctrl + S**.

Paso 17. Hacer un commit cada vez que avance

Con la terminal dentro de mis_trabajos:

```
git status
```

Muestra en rojo lo que cambió o es nuevo.

```
git add .
```

Marca **todos** los cambios para guardarlos (el punto significa «todo lo de esta carpeta»).

```
git commit -m "MR1: corrijo los tres fragmentos con errores"
```

Guarda la «foto». Lo que va **entre comillas** es **su mensaje**: describa en pocas palabras lo que hizo.

```
git log --oneline
```

Muestra la lista de commits que lleva.

Cuándo hacer commit: al terminar cada ejercicio y siempre antes de una entrega.

Para volver a la carpeta principal escriba `cd ...`

9. Problemas frecuentes

Síntoma	Causa probable	Qué hacer
git : El término 'git' no se reconoce	Git no instalado, o abrió Visual Studio Code antes de instalarlo.	Instale Git, cierre Visual Studio Code por completo y ábralo de nuevo.
destination path 'Informacion_programacion' already exists	Ya clonó antes.	No repita el clone; use <code>git -C Informacion_programacion pull</code> .
fatal: not a git repository	La terminal está en una carpeta equivocada.	Escriba <code>pwd</code> y vaya a <code>C:\Programacion</code> .
(.venv) no aparece	No activó el entorno.	<code>.\.venv\Scripts\Activate.ps1</code> estando en <code>C:\Programacion</code> .
«la ejecución de scripts está deshabilitada»	Política de PowerShell.	<code>Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned</code> y responda S .
ModuleNotFoundError: No module named 'pandas'	Bibliotecas no instaladas o cuaderno con otro kernel.	Active <code>.venv</code> , repita el Paso 12 y elija el kernel <code>.venv</code> (Paso 14).
El kernel <code>.venv</code> no aparece	Falta recargar.	Ctrl + Shift + P → Developer: Reload Window ; revise el Paso 11.
FileNotFoundError al ejecutar <code>test_datos.py</code>	Nombre de equipo mal escrito, o terminal fuera de <code>C:\Programacion</code> .	Revise la línea <code>EQUIPO = ...</code> y use <code>pwd</code> .
python abre la tienda de Microsoft	Python no está instalado o falta el PATH.	Reinstale Python marcando «Add python.exe to PATH» (Paso 5).

Síntoma	Causa probable	Qué hacer
No sé en qué carpeta estoy	—	Escriba pwd.

Si nada de esto funciona: tome una **captura de pantalla** de toda la terminal (con el comando que escribió y el mensaje completo) y envíela al docente.

10. Hoja de referencia rápida

Primera vez (en la terminal, dentro de C:\Programacion):

```
git config --global user.name "Nombre Apellido"
git config --global user.email "su_correo@gmail.com"
git clone https://github.com/cguillermo79/Informacion_programacion.git
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r Informacion_programacion\requirements.txt
```

Cada vez que abra el proyecto:

```
.\.venv\Scripts\Activate.ps1
git -C Informacion_programacion pull
```

Cada vez que termine un avance (dentro de mis_trabajos):

```
git status
git add .
git commit -m "Descripción corta de lo que hizo"
```

11. Reglas importantes

- Use **solo la base de su equipo**; es de **solo lectura** y contiene únicamente registros **2019–2024**. Es una **muestra didáctica** del dataset original.
- **No edite** archivos dentro de Informacion_programacion: trabaje en las copias de mis_trabajos.
- **No suba** a GitHub ni comparta las bases ni datos personales.
- **No suba los datos del proyecto** a herramientas externas de inteligencia artificial.
- Si usa IA como apoyo, debe poder **explicar cada línea** de su código.